REMAN · MS COBOL APPLICATION SUPPORT

RCA Agent v2 Architecture

Draft for review · 8 Oct 2026 · a new backend behind the POC's approved UI

A support engineer types an issue or pastes a batch alert. V2 cleans it up, searches pattern cards, past incidents and the AWS Transform docs in one go, and answers with a root cause, resolution steps, preventive actions, evidence and a confidence score it can explain. On the common path it calls the LLM twice. It brings in an agent only when the search is unsure.

69 incidents
knowledge base and regression tests

45 incidents
held out, evaluation only

15 pattern cards
drafted, waiting for SME review

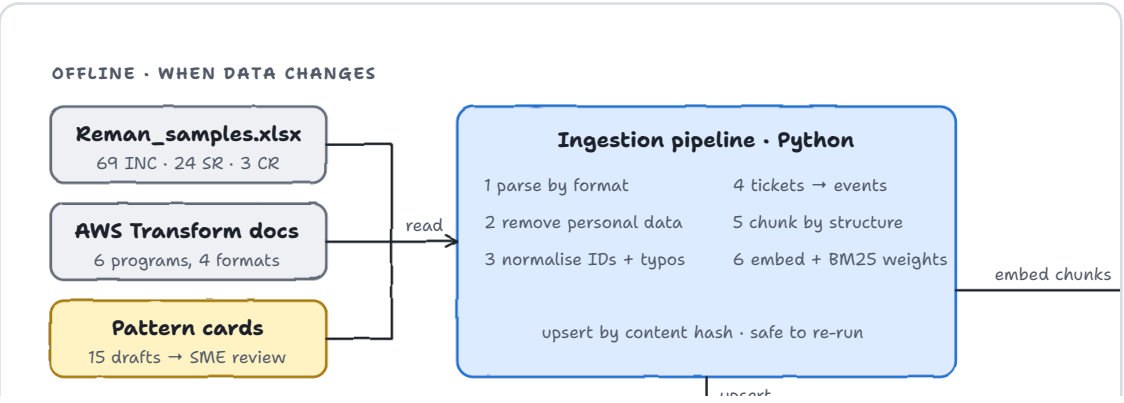
2 LLM calls
on the common path (POC v1 made 6)

Our code, no LLM · LLM call via Azure model-router · Supabase storage · External service · Human input (SME) · User, UI, files · Not loaded

Whole system · One question · Search · Loading knowledge · Database · What the user sees · Evaluation · Stack · Not in the demo · Build order

The whole system

The system has two halves. The offline half turns the client's files into searchable rows in Supabase, and runs again whenever the data changes. The online half answers one question at a time: Streamlit sends the query to a LangGraph pipeline, which reads Supabase and calls Azure OpenAI. LangSmith records every step, and the evaluation harness drives the pipeline from outside.



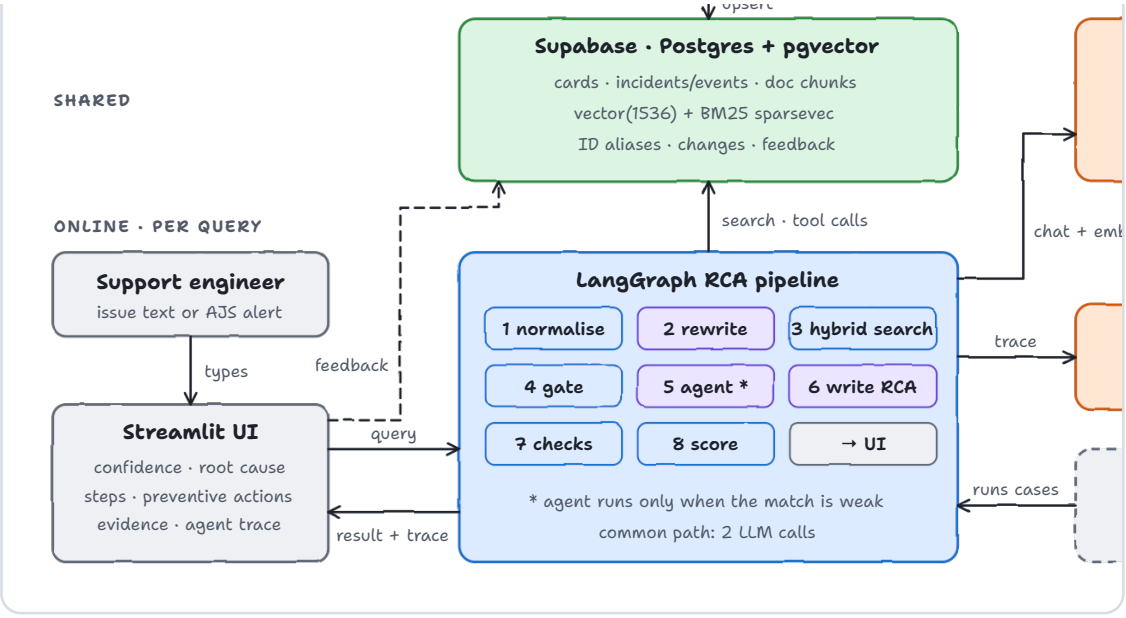


Fig. 1 · Whole system. Supabase is the only store: both halves meet there. The harness file feeds the pipeline from outside and never enters the knowledge base, so evaluation scores stay honest.

Kept from POC v1

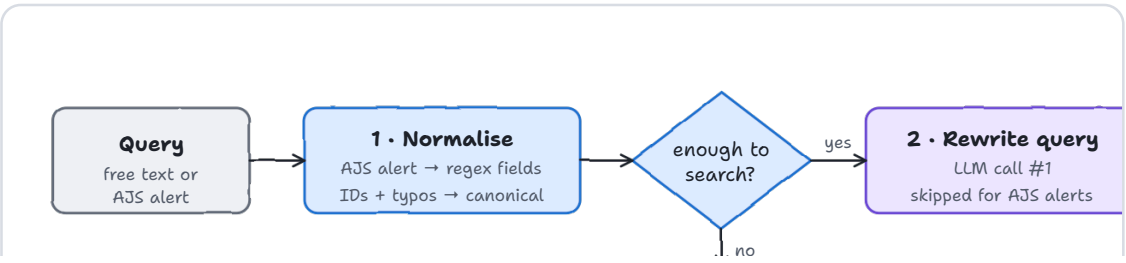
- Streamlit page layout that management approved
- LangGraph orchestration and live progress streaming
- The response shape: root cause, evidence, steps, preventive actions, escalation, confidence
- Program ID map, file typo table, error families
- The choice of which doc files to load and which to skip

Replaced in v2

- Qdrant cloud → Supabase Postgres (dense + BM25 + exact ID in one place)
- Six LLM judgements in a row → two on the common path
- Keyword lists in code → pattern cards and an ID table an SME can review
- LLM-reported confidence → a computed score with reasons
- No regression suite → 69-incident regression run on every change

One question, step by step

Every query goes through the same eight steps. Code handles everything that can be exact, such as alert parsing, ID lookup, search, checks and scoring. The LLM writes the rewrite and the final answer. The agent loop only runs when the gate isn't sure the top match is right.



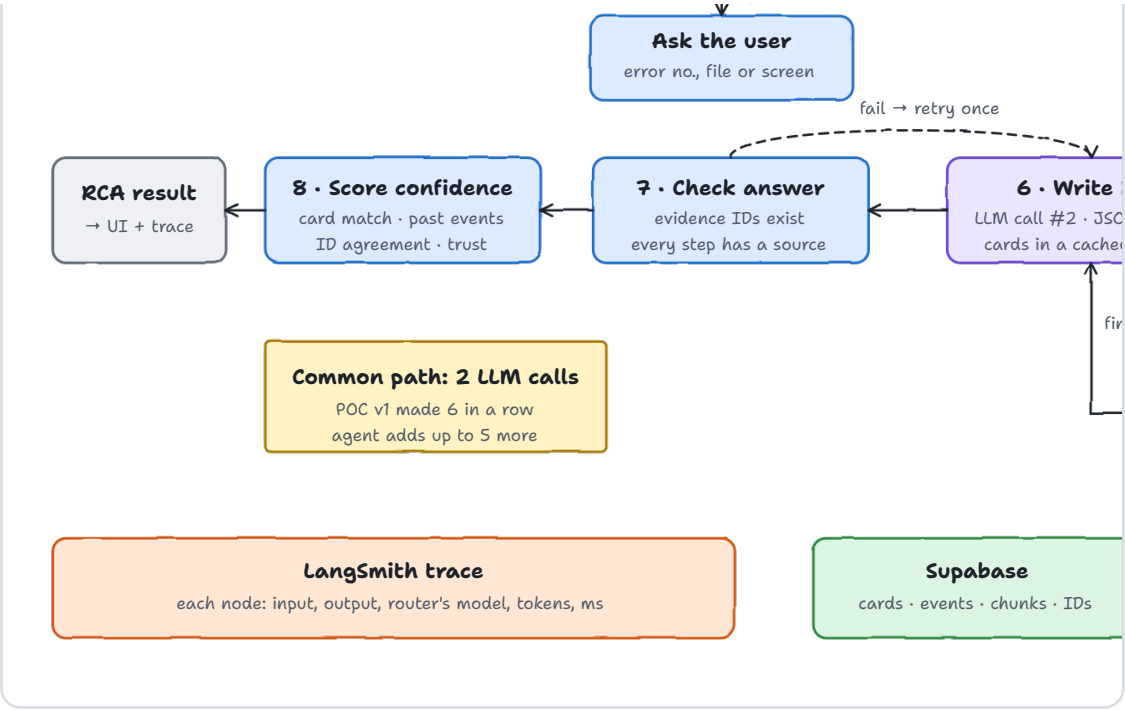


Fig. 2 · The online pipeline. The gate (step 4) decides between the cheap path (straight to step 6) and the agent path (step 5 first). If the checks fail twice, the answer is shown with confidence downgraded to Low and the failed check named, rather than hidden.

STEP	KIND	WHAT IT DOES	WHY IT'S THERE
1 Normalise	code	Parses AJS alerts (abend, job, JCL, step) with a regex; maps IDs and typos to canonical names, e.g. `IN001.MST` → `IN0011.MST`	The IDs decide the answer, so no LLM should be guessing them
Enough to search?	code	Needs a symptom plus at least one of: error number, file, job, program or a recognised application	"LMS error codes" alone can match five different cards
2 Rewrite	LLM #1	Turns free text into {system, symptom, IDs, time}; skipped for alerts, which are already structured	Users write "LMS won open"; cards and tickets use the system's own words
3 Hybrid search	code	Dense, BM25 and exact-ID search in one SQL query, fused by rank (Fig. 3)	Embeddings miss codes; keywords miss paraphrases
4 Gate	code	Strong match means the top hit is a card, it leads the next card by a clear margin, and its IDs or error family agree with the query	Keeps most traffic on the cheap, predictabl path
5 Agent	LLM · ≤5 tools	Looks up files, related incidents, changes and doc sections until it can explain the issue or concludes there is no match	Only for the cases the gate can't settle
6 Write	LLM #2	Fills the response JSON from the	One call with short

RCA		evidence. The card catalogue sits in a fixed prompt prefix so Azure's prompt caching can reuse it	rules, enforced by the schema
7 Check	code	Every cited ID exists; every step is tagged ticket, doc, card or SME; retries once	Replaces the POC's LLM validator with checks that can't drift
8 Score	code	Computes confidence from the evidence (below)	A number the UI can explain

How the confidence score is built

The LLM's own estimate is one input, never the whole score. The score combines:

- **Card match:** rank of the best card and its lead over the runner-up.
- **Past events:** how many separate events back the card. Eleven alerts from one batch failure count once.
- **ID agreement:** whether the error number, file or job in the query matches the card's signals.
- **Source trust:** SME-approved card, then resolved ticket, then technical doc, then generated rules, then the program catalogue.
- **LLM judgement:** High, Medium or Low from step 6.

The starting bands are High ≥ 0.75, Medium 0.45–0.75 and Low below 0.45. They are placeholders: we tune them on the 69 regression incidents, never on the 45. The UI shows the reasons next to the number.

How search finds the evidence

Yes, BM25 is in. It runs as a third search channel next to dense vectors and exact-ID matching, all inside one Postgres query. We compute BM25 weights in Python with our own tokenizer, so codes like `IN0011.MST`, `U4005` and `F8RH0430` stay whole, and store them in pgvector's `sparsevec` column. That needs nothing beyond the pgvector extension Supabase already has.

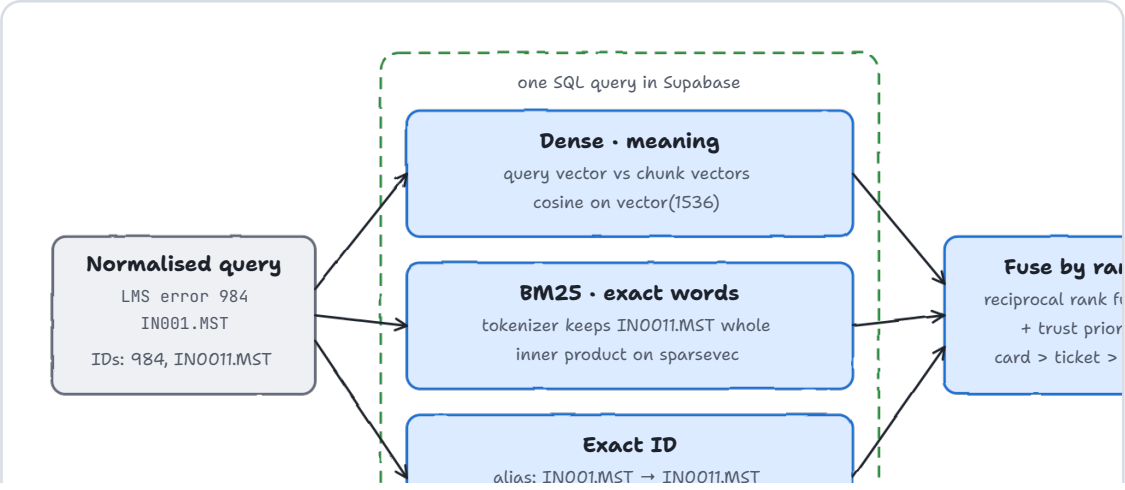




Fig. 3 · Hybrid search. Each channel returns a ranked list, and fusion uses ranks rather than raw scores, so no channel's scale can swamp the others. Collapsing duplicates stops one noisy batch failure from filling all eight slots, as it could in the POC.

BM25 details

- **Tokenizer:** lowercases; keeps `IN0011.MST`, `S0C7` and `JCL=A.F8RH0410` as whole tokens, and also emits their parts (`in0011`, `mst`) so partial mentions still match.
- **Weights:** standard BM25 (k1 = 1.2, b = 0.75), with IDF computed over the whole corpus at load time. The corpus is a few thousand rows, so recomputing it on every load is cheap.
- **Search:** an exact scan on `sparsevec` is fast at this size, so it doesn't need its own index.
- **Why not Postgres full-text search:** its ranking isn't BM25, and its parser splits codes like `IN0011.MST` unpredictably.

How knowledge gets in

One Python pipeline loads everything the agent may cite. Each source is parsed according to its format and chunked along its own structure, never into fixed-size pieces. Files that only duplicate other files are never loaded, and neither is the harness.

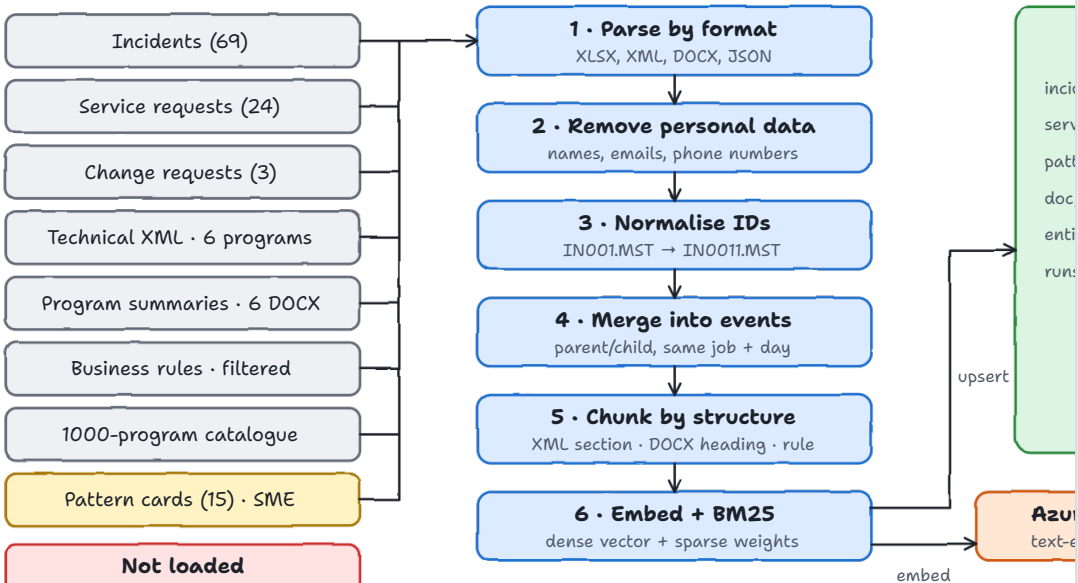


Fig. 4 · Loading knowledge. The red boxes have no arrow out: those files never reach the database.

Pattern cards load as **draft** until the SME review, and the UI labels them that way.

What Supabase holds

Ten tables in one Postgres database. Searchable rows carry both a dense `vector(1536)` and a BM25 `sparsevec` ; IDs are linked through `entity_links` so the exact-ID channel and the agent's tools can join on them.

TABLE	HOLDS	KEY COLUMNS
`incidents`	The 69 sample incidents, parsed into fields	number, event_id, symptom_text, root_cause action_taken, service, opened_at, embeddin bm25
`events`	Duplicate tickets merged into one occurrence	id, parent_number, ticket_count, first_seen, last_seen
`service_requests` , `changes`	SR wording and work notes; CR fix history	number, text, programs, opened_at / closed_at
`pattern_cards`	The 15 cards (draft → approved)	id, status, version, symptoms, signals, root_cause, steps (each with a source tag), embedding, bm25
`card_evidence`	Which incidents back each card	card_id, incident_number
`doc_chunks`	Doc sections from XML, DOCX and filtered rules	source_file, program, section_path, trust_tie role (mechanism / procedure), text, embedding, bm25
`entities` , `entity_aliases`	Every error code, file, job, program and dataset, plus its spellings	type, canonical · alias → entity_id (e.g. IN001.MST → IN0011.MST)
`entity_links`	Which rows mention which IDs	entity_id, item_type, item_id
`runs` , `feedback`	Each query, its answer, and the engineer's verdict	query, result JSON, confidence, trace_url · verdict, actual_fix

What the user sees

The page keeps the POC's approved structure. Each block on screen maps to one field of the response JSON, so the UI and the pipeline can change independently. The example below is illustrative; the values are not real output.

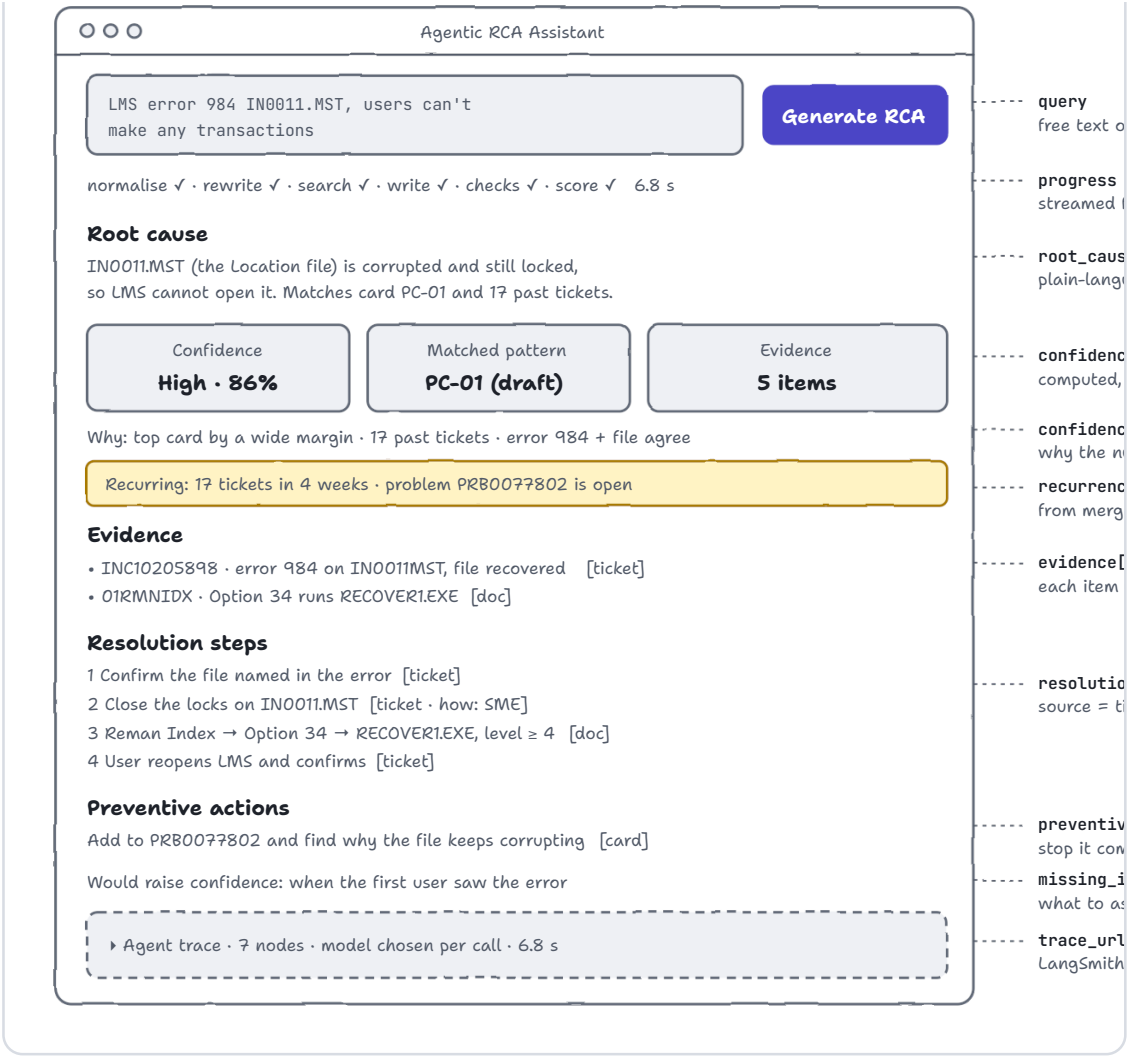


Fig. 5 · The result page. Left: what the engineer sees. Right: the response field behind each block. Every resolution step carries a source tag, so a step nobody has documented shows up as **SME** instead of sounding confident.

How we know it works

The harness file splits into two sets. The 69 incidents also in the knowledge base form the regression suite, run on every change. The 45 held-out incidents are scored only at milestones and are never used to tune anything. The pipeline sees what a support engineer would see; the resolution notes go straight to the judge.





Fig. 6 · Evaluation loop. The dashed path is the guard: resolution notes reach the judge without passing through the pipeline. A change merges only if the regression score doesn't drop.

What we measure

- **Card hit @1 and @3:** is the right pattern card first, or in the top three?
- **Root-cause correctness:** an LLM judge compares the answer with the actual resolution notes; an SME spot-checks a sample.
- **Correct "no close match":** about 13 of the 45 are issue types the cards don't cover, and the right answer there is to say so.
- **Sourced steps:** the share of resolution steps with a source tag. The target is all of them.
- **Calibration:** how often High-confidence answers are correct, so the score means something.
- **Cost and speed:** LLM calls per query, p95 latency, and which models the router chose.

Stack

LAYER	CHOICE	NOTE
UI	Streamlit (from POC v1)	Keeps the approved layout; streams node progress
Orchestration	LangGraph (from POC v1)	The graph in Fig. 2
Chat LLM	Azure OpenAI `model-router`	One deployment; the model it picks is logged on every call
Embeddings	Azure `text-embedding-3-small` · 1536-d	The existing deployment; we change it only if evaluation shows symptom matches being missed
Database	Supabase Postgres + pgvector	`vector` for dense, `sparsevec` for BM25; replaces Qdrant
Tracing	LangSmith	Already configured in the existing `.env`
Ingestion	Python scripts	openpyxl, lxml and the DOCX XML; idempotent upserts
Config	`.env`	Secrets stay out of git

Left out of the first demo

- **SSO and roles:** the demo runs locally and is open. Personal data is still removed at load time.
- **Live ServiceNow and SharePoint connectors:** the demo works from file drops; connectors come once the pipeline is proven.
- **Semantic cache:** too few users to pay off, and a risk of serving a stale answer.
- **Separate reranker model:** rank fusion plus the gate is enough at this corpus size. We add one only if evaluation shows ranking errors.
- **Fine-tuning:** 69 incidents are far too few.
- **Graph database and multi-agent setups:** Postgres joins and one bounded agent cover what we need.

Build order

Measurement comes first, so every later step has a number to beat.

1. **Harness and baseline.** Load the harness, run POC v1 on both sets, record its scores in LangSmith.
2. **Database and ingestion.** Supabase schema; load incidents, events, docs, IDs, and the cards as drafts.
3. **Hybrid search.** The three channels plus fusion; check card hit@3 on the 69.
4. **Pipeline without the agent.** Steps 1–4 and 6–8, with the UI wired to the new response JSON.
5. **Agent loop.** Tools and the five-call limit, for weak matches only.
6. **Confidence and cards.** Calibrate the score bands on the 69; fold in the SME's card review.
7. **Evaluation and demo.** Score the 45, compare with the POC baseline, prepare the demo.

Still open

- SME review of the 15 draft pattern cards.
- Confirming structured JSON output works for every model the router picks.
- Final confidence thresholds, after calibration.